

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN



**BÁO CÁO CHUYÊN ĐỀ
ĐỒ ÁN TỐT NGHIỆP**

**Đề tài: HỆ THỐNG AGENTIC AI BÁN TỰ ĐỘNG SINH
VÀ KIỂM CHỨNG KIỂM THỬ ĐẦU-CUỐI DỰA TRÊN
PLAYWRIGHT**

*Từ tài liệu yêu cầu đến mã Playwright chạy được,
có truy vết, có neo DOM và có con người phê duyệt*

GVHD: [Họ tên giảng viên hướng dẫn]

Nhóm sinh viên thực hiện:

1. Nguyễn Lê Sang

MSSV: 25210178

2. [Họ tên thành viên 2]

MSSV: [...]

Tp. Hồ Chí Minh, 10/2026

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

LỜI MỞ ĐẦU

Kiểm thử đầu-cuối (end-to-end, E2E) kiểm tra một ứng dụng web theo đúng cách người dùng thật sử dụng nó: mở trang, nhập liệu, bấm nút và quan sát kết quả. Viết và duy trì các kịch bản này tốn nhiều công. Kiểm thử viên phải đọc tài liệu yêu cầu, tự suy ra test case, rồi viết mã tự động hoá và sửa lại mỗi khi giao diện thay đổi.

Mô hình ngôn ngữ lớn (LLM) hiện đọc được tài liệu yêu cầu và viết được mã kiểm thử. Khi đưa vào quy trình thật, chúng gặp bốn vấn đề: tài liệu yêu cầu thường mơ hồ nhưng LLM vẫn tự điền chỗ trống; không còn dấu vết nối test với yêu cầu gốc; locator do LLM đoán có thể không tồn tại trên trang; và cơ chế tự sửa lỗi có thể làm yếu assertion để test "xanh" giả. Đồ án này xây dựng một hệ thống Agentic AI bán tự động cho chuỗi từ tài liệu yêu cầu đến mã Playwright Python chạy được. Trong hệ thống, các agent chỉ phân tích, sinh và đề xuất. Con người giữ quyền quyết định ở các công ảnh hưởng tới ý nghĩa kiểm thử.

Nội dung báo cáo gồm sáu chương:

- Chương 1: Tổng quan đề tài
- Chương 2: Cơ sở lý thuyết và nghiên cứu liên quan
- Chương 3: Phân tích, thiết kế và xây dựng hệ thống
- Chương 4: Môi trường thực nghiệm và phương pháp đánh giá
- Chương 5: Kết quả thực nghiệm và đánh giá
- Chương 6: Kết luận và hướng phát triển

Toàn bộ thực nghiệm chỉ chạy trên các website demo công khai được dựng ra để luyện kiểm thử tự động (SauceDemo, The Internet, UI Testing Playground, DemoQA, Practice Test Automation, OrangeHRM Demo). Mọi con số trong báo cáo đều tái lập được bằng lệnh ghi trong Chương 4.

BẢNG PHÂN CÔNG CÔNG VIỆC

Bảng 0.1 Bảng phân công công việc

Thành viên	Vai trò	Công việc chính
Nguyễn Lê Sang	AI / Web / Data	Kiến trúc agent, tích hợp LLM (cloud và local), Requirement Agent, bộ đánh giá RQ1, giao diện Streamlit, thực nghiệm RQ2
[Họ tên thành viên 2]	QA / Playwright	Khảo sát Playwright và locator, tiêu chí đánh giá test case, yêu cầu cho traceability và constrained repair, catalog target RQ2
Cả nhóm		Chốt research gap và MVP, review tài liệu và pull request, ghi biên bản họp

DANH MỤC THUẬT NGỮ

Bảng 0.2 Bảng danh mục thuật ngữ

Thuật ngữ	Giải thích
Kiểm thử đầu-cuối (E2E)	Kiểm thử toàn bộ luồng nghiệp vụ qua giao diện như người dùng thật
Agent (tác tử)	Thành phần dùng LLM để thực hiện một nhiệm vụ có đầu vào, đầu ra theo schema xác định
Locator	Biểu thức xác định phần tử trên trang, ví dụ <code>get_by_role("button", name="Login")</code>
Grounding (neo DOM)	Kiểm chứng locator trên trang thật: phần tử có tồn tại và khớp đúng một hay không
Accessibility tree	Cây trợ năng trình duyệt dựng từ DOM: mỗi nút có role, tên hiển thị và trạng thái
ARIA snapshot	Bản chụp cây trợ năng dạng YAML do Playwright sinh (<code>locator.aria_snapshot()</code>)
Test oracle	Cơ chế quyết định test đạt hay không; ở đây là các <code>assertion expect_*</code>
Self-healing	Tự sửa locator khi phần tử đối tượng thuộc tính mà nghiệp vụ không đổi
Human-in-the-loop	Con người duyệt tại các điểm quyết định thay vì để hệ thống tự vận hành
Boundary Value Analysis	Kỹ thuật thiết kế test chọn giá trị tại và ngay ngoài biên của ràng buộc
First-run pass rate	Tỉ lệ script do LLM sinh chạy đạt ngay lần đầu, không qua sửa
Over-flag	Tỉ lệ yêu cầu sạch bị gán cờ mơ hồ nhầm
Assertion weakening	Làm yếu lệnh kiểm chứng (ví dụ <code>toBe(5)</code> thành <code>toBeTruthy()</code>) để test đạt giả

DANH MỤC TỪ VIẾT TẮT

Bảng 0.3 Bảng danh mục từ viết tắt

Từ viết tắt	Viết đầy đủ	Ý nghĩa
AG	Approval Gate	Cổng phê duyệt của con người
ARIA	Accessible Rich Internet Applications	Chuẩn W3C mô tả vai trò, tên của phần tử cho công nghệ hỗ trợ
BVA	Boundary Value Analysis	Phân tích giá trị biên
DOM	Document Object Model	Cây đối tượng của trang HTML
DSL	Domain-Specific Language	Tập action Playwright rút gọn mà LLM được phép sinh
E2E	End-to-End	Kiểm thử đầu-cuối
HITL	Human-in-the-loop	Con người trong vòng lặp
LLM	Large Language Model	Mô hình ngôn ngữ lớn
RQ	Research Question	Câu hỏi nghiên cứu
SRS	Software Requirements Specification	Đặc tả yêu cầu phần mềm
TC	Test Case	Cá kiểm thử
UI	User Interface	Giao diện người dùng

DANH MỤC CÁC BẢNG, HÌNH ẢNH

Danh mục các bảng:

Bảng 0.1 Bảng phân công công việc	4
Bảng 0.2 Bảng danh mục thuật ngữ	5
Bảng 0.3 Bảng danh mục từ viết tắt	6
Bảng 1.1 Bốn câu hỏi nghiên cứu và trạng thái trong chuyên đề.....	13
Bảng 1.2 Công nghệ sử dụng	14
Bảng 2.1 Đối chiếu năng lực 12 nguồn với nhu cầu đề tài	19
Bảng 3.1 Các thư mục mã nguồn và vai trò	21
Bảng 3.2 Các trường của StructuredRequirement.....	22
Bảng 3.3 Rubric bảy loại mơ hồ.....	22
Bảng 3.4 DSL action của PlaywrightPlan	23
Bảng 3.5 Phân mức rủi ro của đề xuất sửa	25
Bảng 3.6 Các cổng phê duyệt	25
Bảng 4.1 Các website trong catalog RQ2.....	27
Bảng 5.1 Kết quả phát hiện mơ hồ trên tập 15 yêu cầu.....	30
Bảng 5.2 Ảnh hưởng của việc siết prompt (trước → sau).....	30
Bảng 5.3 F1 theo từng loại mơ hồ	30
Bảng 5.4 First-run pass rate theo website, hai nhánh.....	31
Bảng 5.5 Bảng chéo kết quả theo cặp target	31
Bảng 5.6 Nguyên nhân thất bại theo nhánh.....	32
Bảng 5.7 Hai lần đo RQ2 trước và sau khi sửa đặc tả goto.....	32
Bảng A.1 Catalog target RQ2 (datasets/reference/rq2_targets.json)	38

Danh mục các hình:

Hình 3.1 Kiến trúc tổng quan của hệ thống.....	21
--	----

Hình 3.2	Luồng pipeline và các cổng phê duyệt	21
Hình 4.1	Quy trình đo RQ2 trên hai nhánh	28
Hình 5.1	Tỉ lệ đạt lần đầu theo website, baseline và DOM-aware.....	31

MỤC LỤC

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN	2
LỜI MỞ ĐẦU	3
BẢNG PHÂN CÔNG CÔNG VIỆC	4
DANH MỤC THUẬT NGỮ	5
DANH MỤC TỪ VIẾT TẮT	6
DANH MỤC CÁC BẢNG, HÌNH ẢNH	7
Chương 1: TỔNG QUAN ĐỀ TÀI	12
1.1 Đặt vấn đề và lý do chọn đề tài	12
1.2 Mục tiêu của đề tài	12
1.3 Câu hỏi nghiên cứu	13
1.4 Phạm vi nghiên cứu	13
1.4.1 Trong phạm vi	13
1.4.2 Giới hạn của đề tài	13
1.5 Đóng góp chính	14
1.6 Công nghệ sử dụng	14
Chương 2: CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN	16
2.1 Kiểm thử đầu-cuối và Playwright	16
2.1.1 Kiểm thử đầu-cuối	16
2.1.2 Playwright và chính sách locator	16
2.1.3 Accessibility tree và ARIA snapshot	16
2.2 Mô hình ngôn ngữ lớn và hệ tác tử	17
2.2.1 Structured output	17
2.2.2 Agent và con người trong vòng lặp	17
2.3 Kỹ thuật thiết kế test case	17
2.4 Truy vết yêu cầu	17

2.5 Self-healing và bài toán oracle	18
2.6 Chỉ số đánh giá	18
2.7 Các nghiên cứu liên quan	18
Chương 3: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG	20
3.1 Yêu cầu hệ thống	20
3.1.1 Yêu cầu chức năng	20
3.1.2 Yêu cầu phi chức năng	20
3.2 Kiến trúc tổng quan	20
3.3 Requirement Analysis Agent.....	22
3.4 Test Design Agent và Boundary Value Analysis	23
3.5 Playwright Generation Agent	23
3.5.1 DSL action rút gọn	23
3.5.2 Chính sách locator và grounding	24
3.5.3 Ngữ cảnh accessibility tree (nhánh DOM-aware)	24
3.6 Execution Agent	24
3.7 Repair Agent và chính sách sửa có ràng buộc.....	24
3.8 Truy vết, công duyệt và giao diện	25
3.9 Lớp trừu tượng LLM	26
3.10 Kiểm thử phần mềm	26
Chương 4: MÔI TRƯỜNG THỰC NGHIỆM VÀ PHƯƠNG PHÁP ĐÁNH GIÁ	27
4.1 Môi trường thực nghiệm.....	27
4.2 Đánh giá RQ1: phát hiện mơ hồ.....	27
4.3 Đánh giá RQ2: ngữ cảnh DOM và first-run pass rate	27
4.3.1 Catalog target.....	27
4.3.2 Oracle viết tay: kiểm chứng target khả thi	28
4.3.3 Hai nhánh so sánh.....	28
4.3.4 Kiểm soát nhiễu hạ tầng	29

4.4 Đánh giá RQ3 và RQ4.....	29
Chương 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	30
5.1 RQ1: phát hiện mơ hồ trong yêu cầu.....	30
5.2 RQ2: ngữ cảnh DOM và first-run pass rate.....	31
5.2.1 Lần đo thứ nhất: một lỗi đặc tả prompt bị che khuất	32
5.2.2 Thảo luận	32
5.3 RQ3: sửa lỗi có ràng buộc (demo)	33
Chương 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	35
6.1 Kết quả đạt được.....	35
6.2 Hạn chế	35
6.3 Hướng phát triển.....	36
TÀI LIỆU THAM KHẢO	37
PHỤ LỤC A: DANH SÁCH 50 TARGET RQ2	38

Chương 1: TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề và lý do chọn đề tài

Trong quy trình phát triển phần mềm, kiểm thử E2E là chốt chặn cuối trước khi phát hành. Việc chuyển thủ công tài liệu yêu cầu (SRS, use case, user story) thành kịch bản và mã kiểm thử tốn thời gian và khó bảo trì. Một nghiên cứu công nghiệp quy tới 73% thất bại của test suite về lỗi locator chứ không phải lỗi chức năng [6]. Các nghiên cứu 2024–2026 cho thấy LLM sinh được test case từ tài liệu yêu cầu [5][7][11] và thao tác được trên web thật [10]. Khi áp dụng thực tế, nhóm nhận thấy bốn vấn đề mà chưa nguồn nào trong 12 nguồn khảo sát giải quyết trọn vẹn:

- Yêu cầu chưa được quản trị như một artifact có cấu trúc. Các công cụ thường đưa nguyên tài liệu cho LLM, giả định đầu vào đầy đủ và không có bước phát hiện yêu cầu thiếu, mơ hồ hoặc mâu thuẫn [4][7]. LLM vì thế tự điền giả định vào expected result.
- Truy vết bị đứt. Ma trận truy vết thường nằm trong Excel hoặc hệ thống ALM tách khỏi mã, và khoảng 80% người hành nghề coi chi phí cập nhật thủ công là rào cản chính [9]. Khi LLM tự sinh test, không ai biết đoạn mã nào phục vụ yêu cầu nào.
- Locator do LLM đoán có thể không tồn tại. Không neo vào DOM thật, mô hình suy ra selector từ mẫu đã thấy khi huấn luyện; một case study ghi nhận agent bỏ qua công cụ kiểm chứng DOM và gây hàng loạt lỗi timeout [8].
- Tự sửa lỗi không ràng buộc làm hỏng oracle. Trên 300 báo cáo thực thi, một hệ multi-agent chỉ hội tụ thật khoảng 50% thay vì 70% bề mặt, vì agent làm yếu assertion hoặc xóa test đang lỗi để đạt trạng thái Pass [8].

Đề tài chọn hướng bán tự động: LLM làm phần nặng (phân tích, sinh, đề xuất), còn các quyết định làm thay đổi ý nghĩa kiểm thử phải qua con người. Hướng này khớp với khuyến nghị của [8] về giới hạn vòng sửa, phát hiện trôi ngữ nghĩa và bắt buộc phê duyệt thay đổi assertion.

1.2 Mục tiêu của đề tài

- Xây dựng pipeline năm agent: Requirement Analysis, Test Design, Playwright Generation, Execution và Repair, nối từ tài liệu yêu cầu tới script Playwright Python chạy trên website thật.

- Chuẩn hoá yêu cầu thành schema có cấu trúc và gắn cờ mơ hồ theo rubric, chặn ở cổng duyệt trước khi sinh test.
- Duy trì truy vết từ yêu cầu tới test case, script, kết quả chạy và đề xuất sửa.
- Giới hạn tự sửa lỗi bằng chính sách rủi ro tất định (không do LLM tự đánh giá) và ngân sách sửa.
- Đo định lượng: độ chính xác phát hiện mơ hồ (RQ1) và tỉ lệ script chạy đạt ngay lần đầu khi có và không có ngữ cảnh DOM (RQ2).

1.3 Câu hỏi nghiên cứu

Bảng 1.1 Bốn câu hỏi nghiên cứu và trạng thái trong chuyên đề

RQ	Câu hỏi	Trạng thái
RQ1	Chuẩn hoá tài liệu yêu cầu thành biểu diễn có cấu trúc có cải thiện tính hợp lệ và độ bao phủ của test case so với đưa thẳng tài liệu cho LLM không?	Đã đo phần phát hiện mơ hồ (sơ bộ)
RQ2	Kết hợp yêu cầu với DOM hoặc accessibility tree có cải thiện tỉ lệ mã Playwright chạy đạt ở lần đầu không?	Đã đo trên 50 target, 6 site
RQ3	Sửa lỗi có ràng buộc và phê duyệt của con người có tăng tỉ lệ sửa thành công mà không đổi ý nghĩa test case không?	Có cơ chế và demo, chưa đo trên tập lớn
RQ4	Hệ thống giảm bao nhiêu thời gian và thao tác thủ công so với quy trình truyền thống?	Chưa đo

1.4 Phạm vi nghiên cứu

1.4.1 Trong phạm vi

Đề tài nhằm vào kiểm thử chức năng E2E cho ứng dụng web, sinh mã Playwright bằng Python. Đầu vào là tài liệu DOCX, PDF dạng văn bản, Markdown hoặc text, chứa use case, user story kèm acceptance criteria hoặc functional requirement. Hệ thống sinh test case positive, negative, boundary, error guessing và alternative flow; chạy test, thu bằng chứng; đề xuất sửa trong ràng buộc và có con người phê duyệt.

1.4.2 Giới hạn của đề tài

- Không kiểm thử mobile, desktop, hiệu năng, bảo mật hay accessibility toàn diện.
- Không sinh đồng thời Selenium, Cypress và Playwright; chỉ Playwright Python, trình duyệt Chromium, chạy tuần tự.
- Không tự sửa mã nguồn của ứng dụng được kiểm thử và không tự suy ra oracle từ yêu cầu mơ hồ.

- Không huấn luyện hay fine-tune LLM; chỉ dùng mô hình có sẵn qua API tương thích OpenAI.
- Lưu trữ bền vào SQL Server và cổng duyệt AG-03..05 trên giao diện web chưa hoàn thành ở thời điểm viết chuyên đề.

1.5 Đóng góp chính

Đề tài không tuyên bố phát minh từng thành phần riêng lẻ. Phần mới nằm ở cách ghép và kiểm soát chúng trong một pipeline Playwright Python:

- Requirement Analysis Agent với rubric bảy loại mơ hồ, chỉ phát hiện mà không tự giải quyết, chờ duyệt ở AG-01. Cột "ambiguity detection" trống trên cả 12 nguồn khảo sát.
- Sinh test biên và âm bản tất định bằng Boundary Value Analysis từ ràng buộc số, bổ sung cho kết quả của LLM.
- Grounding đa bước: đi theo luồng và kiểm chứng locator trên DOM tại đúng bước đó, bắt được phần tử chỉ xuất hiện sau điều hướng.
- Repair policy tất định phân mức rủi ro theo nội dung thay đổi (locator, dữ liệu, assertion, bước), ngân sách sửa bằng 2 và self-healing locator từ DOM sống, mọi đề xuất đều cần người duyệt.
- Bộ đánh giá có ground truth: RQ1 trên tập yêu cầu chèn khuyết tật có chủ đích; RQ2 trên catalog 50 target có oracle viết tay chứng minh mỗi target làm được.

1.6 Công nghệ sử dụng

Bảng 1.2 Công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Ngôn ngữ	Python 3.14	Toàn bộ hệ thống và script kiểm thử sinh ra
Tự động hoá trình duyệt	Playwright 1.62 (sync API), Chromium	Grounding, aria snapshot, chạy script
Mô hình dữ liệu	Pydantic 2	Schema đầu ra của agent, validate JSON
LLM cloud	DeepSeek (deepseek-chat) qua API tương thích OpenAI	Backend chính cho thực nghiệm
LLM local	LM Studio + google/gemma-4-12b (GPU 8 GB)	Backend miễn phí, chạy offline
Sinh mã	Jinja2	Render PlaywrightPlan thành script Python tự chứa
Đọc tài liệu	python-docx, PyMuPDF	Trích văn bản DOCX/PDF, giữ mốc

Thành phần	Công nghệ	Vai trò
		trang để truy vết
Giao diện	Streamlit	Bảng duyệt yêu cầu (AG-01) và test case (AG-02)
Kiểm thử	pytest	Unit test offline bằng LLM giả lập

Chương 2: CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN

2.1 Kiểm thử đầu-cuối và Playwright

2.1.1 Kiểm thử đầu-cuối

Kiểm thử E2E chạy ứng dụng hoàn chỉnh qua giao diện, mô phỏng hành vi người dùng từ đầu tới cuối một luồng nghiệp vụ. Loại kiểm thử này bắt được lỗi tích hợp mà unit test bỏ sót, đổi lại chậm và dễ gây khi giao diện thay đổi. Mỗi test gồm chuỗi bước (mở trang, nhập, bấm) và một hoặc nhiều assertion đóng vai oracle.

2.1.2 Playwright và chính sách locator

Playwright là thư viện tự động hoá trình duyệt của Microsoft, hỗ trợ Chromium, Firefox và WebKit. Hai cơ chế giúp test ổn định hơn. Thứ nhất là auto-wait: thao tác chờ phần tử sẵn sàng, assertion expect() tự thử lại đến hết timeout. Thứ hai là locator hướng người dùng [2]. Tài liệu chính thức khuyến nghị ưu tiên get_by_role, get_by_label, get_by_placeholder, get_by_text và get_by_test_id, hạn chế CSS/XPath vì chúng bám vào cấu trúc DOM dễ đổi. Locator ở chế độ strict: nếu khớp nhiều hơn một phần tử, thao tác báo lỗi. Hệ thống tận dụng điều này để đo chất lượng locator bằng số phần tử khớp.

Từ 2025, Playwright có bộ Test Agents gồm Planner, Generator và Healer [1]: Planner khám phá ứng dụng và viết kế hoạch Markdown, Generator chuyển thành test, Healer chạy lại và vá test hỏng. Bộ này là baseline sản phẩm gần nhất của đề tài nhưng không có bước phát hiện mơ hồ, không có cổng duyệt bắt buộc và chưa công bố chính sách sửa chi tiết.

2.1.3 Accessibility tree và ARIA snapshot

Trình duyệt dựng accessibility tree từ DOM cho công nghệ hỗ trợ (trình đọc màn hình). Mỗi nút có role (button, textbox, link, heading...), tên hiển thị (accessible name) và trạng thái. Cây này gọn hơn DOM nhiều lần vì bỏ các thẻ trình bày, và nó dùng đúng ngôn ngữ của get_by_role. Playwright từ bản 1.49 có locator.aria_snapshot() trả cây này dạng YAML, ví dụ:

```
- heading "Dynamic Controls" [level=4]
- checkbox
- text: A checkbox
- button "Remove"
```

```
- textbox [disabled]
- button "Enable"
```

Đây là ngữ cảnh DOM mà nhánh DOM-aware của RQ2 đưa cho LLM. Hướng ally-first cũng là lỗi của [6], còn [10] lọc DOM rồi đánh chỉ số phần tử cho LLM chọn.

2.2 Mô hình ngôn ngữ lớn và hệ tác tử

2.2.1 Structured output

LLM sinh văn bản tự do, trong khi pipeline cần dữ liệu có cấu trúc để validate và lưu. Các API tương thích OpenAI hỗ trợ response_format kiểu json_schema: máy chủ ép đầu ra theo ngữ pháp sinh từ schema, nên mô hình không thể trả sai kiểu hay thiếu trường. Hệ thống khai báo schema bằng Pydantic, gửi kèm response_format và validate lại phía client.

2.2.2 Agent và con người trong vòng lặp

Trong đồ án, agent là một bước có đầu vào và đầu ra theo schema, dùng LLM ở phần cần suy luận và dùng code tắt định ở phần còn lại. Mô hình human-in-the-loop đặt con người tại các điểm có rủi ro ngữ nghĩa: duyệt yêu cầu đã chuẩn hoá, duyệt test case, duyệt mọi đề xuất sửa đổi assertion, expected result hay bước kiểm thử. Bằng chứng ở [8] cho thấy thiếu các công này thì agent tối ưu cho trạng thái Pass thay vì cho tính đúng.

2.3 Kỹ thuật thiết kế test case

Kiểm thử hộp đen dựa trên đặc tả có các kỹ thuật kinh điển trong chuẩn ISO/IEC/IEEE 29119-4: phân hoạch tương đương, phân tích giá trị biên, đoán lỗi và kiểm thử luồng thay thế. Với ràng buộc "mật khẩu 8–20 ký tự", BVA chọn các giá trị 8 và 20 (biên hợp lệ) cùng 7 và 21 (ngay ngoài biên, ca âm bản). [7] dùng các kỹ thuật này trong prompt; đồ án tách BVA ra thành code tắt định để bảo đảm luôn có ca biên, không phụ thuộc LLM nhớ ra.

2.4 Truy vết yêu cầu

Requirements traceability là khả năng đi xuôi từ một yêu cầu tới các artifact hiện thực nó và đi ngược lại. ReqToCode [9] nhúng phần tử truy vết vào mã để lỗi truy vết thành lỗi biên dịch, nhưng chỉ bảo đảm sự hiện diện cấu trúc chứ không bảo đảm đúng ngữ nghĩa, và không bao phủ kết quả chạy hay đề xuất sửa. Đồ án dùng chuỗi ID tắt định

REQ-001 → REQ-001-TC-01 → script → kết quả → đề xuất sửa để tính độ phủ và tìm artifact mô côi.

2.5 Self-healing và bài toán oracle

Self-healing locator tìm phần tử thay thế khi locator cũ gãy. Similo và HybridSimilo [12] chấm điểm ứng viên trên nhiều thuộc tính (tag, id, class, text, aria-label, vị trí) và đặt tỉ lệ định vị lại cao trên benchmark của họ. [6] chỉ trích lại đúng selector hỏng từ accessibility tree, không tốn lời gọi API. Các kỹ thuật này sửa được phần kỹ thuật nhưng không trả lời được câu hỏi khi nào thay đổi là hợp lệ về nghiệp vụ. Khi agent được phép sửa cả assertion, nó có thể sửa oracle cho khớp với lỗi [8]. Ranh giới mà đồ án chọn: locator tương đương là thay đổi kỹ thuật; assertion, expected result và bước kiểm thử là thay đổi ngữ nghĩa và phải qua người duyệt.

2.6 Chỉ số đánh giá

- Precision = $TP / (TP + FP)$: trong các cờ mờ hồ hệ thống gắn, bao nhiêu phần đúng. Recall = $TP / (TP + FN)$: trong các khuyết tật thật, hệ thống bắt được bao nhiêu. F1 là trung bình điều hoà của hai chỉ số.
- Micro-F1 gộp TP/FP/FN của mọi loại rồi tính; macro-F1 tính F1 từng loại rồi lấy trung bình, nên loại hiếm có trọng số ngang loại phổ biến.
- Over-flag: tỉ lệ yêu cầu sạch (không có khuyết tật chèn) bị gắn cờ. Chỉ số này đo gánh nặng duyệt nhằm cho con người.
- First-run pass rate: số script đạt ngay lần chạy đầu chia cho số target. Script không được sửa tay hay qua Repair Agent.
- Kiểm định McNemar: hai nhánh chạy trên cùng target nên kết quả là dữ liệu cặp. McNemar chỉ dùng các cặp bất đồng (nhánh này đạt, nhánh kia trượt); với cỡ mẫu nhỏ dùng dạng chính xác theo phân phối nhị thức.

2.7 Các nghiên cứu liên quan

Nhóm khảo sát 12 nguồn chia năm mảng: baseline sản phẩm, requirement → test bằng LLM, traceability, DOM grounding và self-healing có ràng buộc. Bảng 2.1 tóm tắt năng lực của các nguồn theo tám tiêu chí mà đề tài cần.

Bảng 2.1 Đối chiếu năng lực 12 nguồn với nhu cầu đề tài

Nguồn	Yêu cầu cấu trúc	Phát hiện mơ hồ	Truy vết	Neo DOM	Oracle	Duyệt người	Sửa ràng buộc
[1] Playwright Test Agents	Một phần	Không	Một phần	Có	Một phần	Không	Một phần
[2] Codegen/locator	Không	Không	Không	Có	Một phần	Một phần	Không
[3] RBTG Survey	Một phần	Một phần	Một phần	Không	Một phần	Một phần	Không
[4] WebTestPilot	Có	Không	Có	Có	Có	Không	Một phần
[5] LLM Scenario Tool	Một phần	Không	Không	Không	Một phần	Một phần	Không
[6] Zero-Cost Self-Healing	Không	Không	Một phần	Có	Một phần	Không	Có
[7] GHF (ISO 29119-4)	Một phần	Không	Một phần	Không	Một phần	Một phần	Không
[8] Autonomous Repair Limits	Một phần	Không	Một phần	Một phần	Không	Một phần	Có
[9] ReqToCode	Có	Không	Có	Không	Không	Một phần	Không
[10] Steward	Không	Không	Không	Có	Một phần	Không	Một phần
[11] System TC (ChatGPT)	Có	Không	Một phần	Không	Một phần	Có	Không
[12] Web Element Relocalization	Không	Không	Không	Có	Không	Không	Có
Nhu cầu đề tài	Có	Có	Có	Có	Có	Có	Có

Cột phát hiện mơ hồ không có nguồn nào đạt "Có". [3] chỉ nêu đây là thách thức mở; [4] giả định đặc tả tự chứa và đầy đủ; [11] ghi nhận LLM hiểu hành vi hệ thống hạn chế khi chỉ đọc SRS nhưng không có bộ phát hiện. Không nguồn nào phủ đủ tám tiêu chí. Mỗi nguồn mạnh ở vài mảnh, nên khoảng trống mà đề tài nhắm tới là sự tích hợp có kiểm soát: một pipeline bán tự động có truy vết, có neo DOM, và ngăn self-healing làm đổi oracle khi chưa được duyệt.

Về RQ2, [10] báo cáo tỉ lệ hoàn thành tác vụ end-to-end 40% với GPT-4 khi có lọc DOM và screenshot, còn [8] cho thấy chiều ngược lại: khi agent bỏ qua kiểm chứng DOM, selector ảo giác gây lỗi dây chuyền. Chưa nguồn nào trong bộ khảo sát so sánh có kiểm soát cùng một tập test case khi có và không có ngữ cảnh accessibility tree. Thực nghiệm RQ2 của đồ án làm đúng phép so sánh này.

Chương 3: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1 Yêu cầu hệ thống

3.1.1 Yêu cầu chức năng

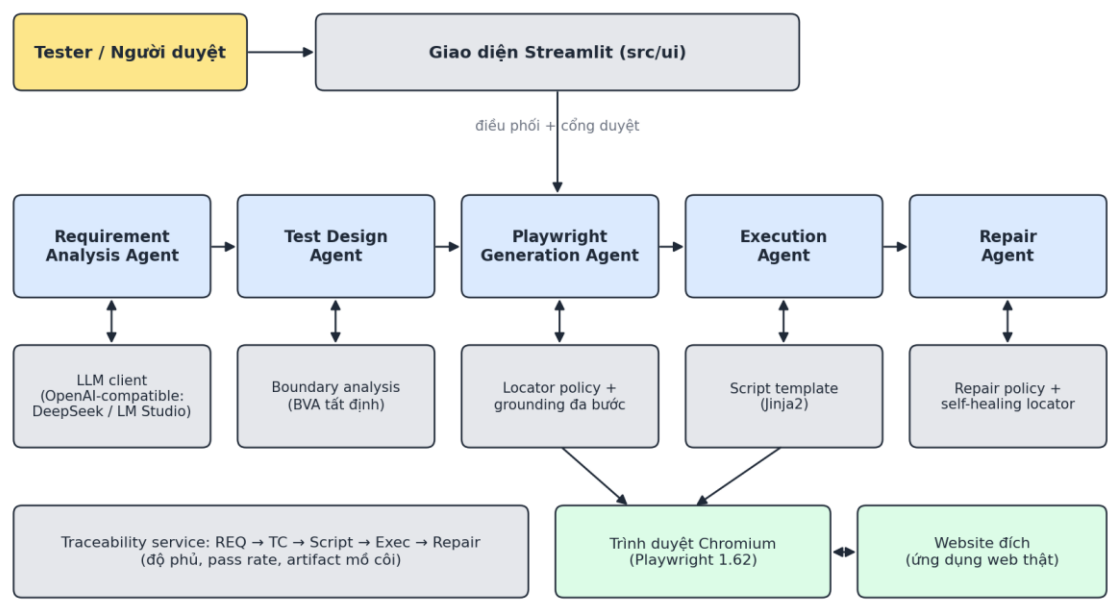
- Nạp tài liệu DOCX, PDF, Markdown, text; trích yêu cầu có cấu trúc kèm trích đoạn nguồn.
- Gắn cờ yêu cầu thiếu, mơ hồ, mâu thuẫn và chờ người duyệt trước khi sinh test (AG-01).
- Sinh test case nhiều loại từ yêu cầu đã duyệt, cho người duyệt hoặc loại bỏ (AG-02).
- Sinh script Playwright Python từ test case, kiểm chứng locator trên trang thật.
- Chạy script, thu ảnh chụp, phân loại passed / failed / error / blocked.
- Khi test lỗi: phân mức rủi ro đề xuất sửa, tự chữa locator từ DOM sống, chờ người duyệt (AG-03..05).
- Dựng chuỗi truy vết, tính độ phủ yêu cầu, độ phủ thực thi, pass rate và artifact mô côi.

3.1.2 Yêu cầu phi chức năng

- Fail-closed: đầu ra sai schema, thiếu bằng chứng hay còn mơ hồ chưa xử lý thì dừng ở trạng thái chờ duyệt.
- Đổi backend LLM (cloud hoặc local) chỉ bằng cách đổi file cấu hình, không sửa agent.
- Kiểm thử offline: unit test không cần mạng, trình duyệt hay API key.
- Không lưu khoá API trong mã nguồn; .env và .env.cloud bị loại khỏi git.
- Tái lập được: mọi số liệu báo cáo chạy lại được bằng lệnh có ghi trong tài liệu.

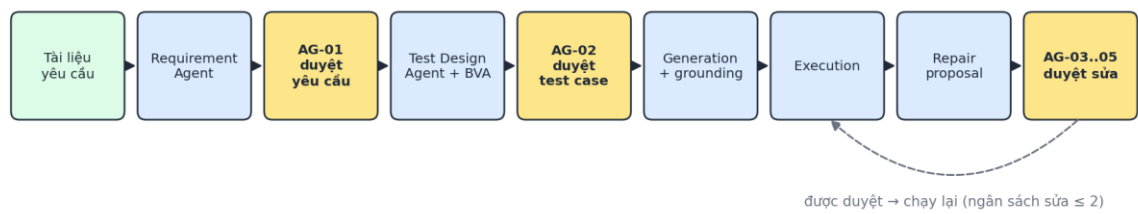
3.2 Kiến trúc tổng quan

Hệ thống gồm năm agent nối tiếp, mỗi agent dựa trên một dịch vụ tắt định (Hình 3.1). Các agent không gọi nhau trực tiếp mà trao đổi qua các model Pydantic. Approval không phải công cụ mà agent tự gọi được: quyết định duyệt chỉ đến từ người dùng qua giao diện. Thiết kế này tránh việc agent tự hợp thức hoá đề xuất của chính nó.



Hình 3.1 Kiến trúc tổng quan của hệ thống

Luồng xử lý đầy đủ đi qua các cổng duyệt như Hình 3.2. Khi test lỗi, Repair Agent tạo đề xuất; đề xuất được duyệt thì script mới được chạy lại; vượt ngân sách sửa thì chuyển sang trạng thái chờ người xem xét.



Ô vàng = cổng con người quyết định; agent chỉ để xuất, không tự áp dụng thay đổi.
Mỗi artifact mang ID truy vết REQ-xxx → REQ-xxx-TC-yy → script → kết quả → để xuất sửa.

Hình 3.2 Luồng pipeline và các cổng phê duyệt

Bảng 3.1 Các thư mục mã nguồn và vai trò

Thư mục / tệp	Vai trò
src/models/	Model Pydantic: Requirement, TestCase, PlaywrightPlan, ExecutionResult, RepairProposal, Approval
src/agents/	Năm agent: requirement, test_design, playwright_generation, execution, repair
src/services/	Dịch vụ tất định: llm_client, boundary_analysis, locator_policy, locator_healing, repair_policy, script_template, traceability, document_loader
src/evaluation/	Bộ đánh giá RQ1: dataset, metrics, evaluator, design_metrics
src/ui/	Giao diện Streamlit (app.py) và logic duyệt thuần (review.py)
run_*.py	CLI cho từng agent, pipeline đầy đủ và thực nghiệm RQ2

Thư mục / tệp	Vai trò
datasets/reference/	Ground truth RQ1 (5 tài liệu) và catalog 50 target RQ2
tests/unit/	Unit test offline

3.3 Requirement Analysis Agent

Agent nhận văn bản do bộ đọc tài liệu trích ra. Với PDF, bộ đọc chèn mốc [trang N] để mỗi yêu cầu truy được về trang gốc. LLM trả về danh sách StructuredRequirement theo schema ở Bảng 3.2. Hệ thống tự đánh ID REQ-001, REQ-002... theo thứ tự và bỏ qua ID do LLM đặt, vì ID của LLM không ổn định giữa các lần chạy. Tài liệu rỗng trả kết quả rỗng mà không gọi LLM.

Bảng 3.2 Các trường của StructuredRequirement

Trường	Ý nghĩa
id	REQ-xxx do hệ thống gán, dùng làm gốc truy vết
title, actor, action	Tên yêu cầu, tác nhân thực hiện, hành động
precondition, expected_outcome	Điều kiện trước và kết quả mong đợi; để trống nếu tài liệu không nói
constraints	Ràng buộc (độ dài, khoảng giá trị...), đầu vào cho BVA
source_excerpt	Trích đoạn nguồn nguyên văn
ambiguities	Danh sách AmbiguityFinding: loại, mô tả, trích đoạn, gợi ý, mức độ

Rubric mơ hồ có bảy loại (Bảng 3.3). Mâu thuẫn giữa nhiều yêu cầu được đặt ở global_ambiguities vì nó không thuộc riêng yêu cầu nào. Agent chỉ phát hiện, không tự làm đầy chỗ thiếu: yêu cầu bị gán cờ dừng ở AG-01 cho tới khi người dùng làm rõ hoặc chấp nhận.

Bảng 3.3 Rubric bảy loại mơ hồ

Loại	Dấu hiệu	Ví dụ
missing_actor	Không rõ ai thực hiện	"Đơn hàng được hủy trong 24 giờ"
missing_precondition	Thiếu điều kiện trước cần thiết	Đổi mật khẩu mà không nói phải đăng nhập
vague_quantifier	Lượng từ không đo được	"phản hồi nhanh", "một vài sản phẩm"
missing_expected_outcome	Không nói kết quả mong đợi	"Người dùng bấm Thanh toán" rồi dừng
underspecified_action	Hành động thiếu tham số	"Nhập thông tin hợp lệ"
conflict	Hai yêu cầu mâu thuẫn	"mật khẩu 8–20 ký tự" và "tối thiểu 18 ký tự"
other	Mơ hồ khác	

Một lỗi đáng ghi lại: khi bỏ phần mô tả schema khỏi prompt để tiết kiệm khoảng 35% token đầu vào, Gemma dồn mọi cờ mơ hồ vào `global_ambiguities` thay vì gắn theo từng yêu cầu. Micro-F1 rơi xuống 0,07. Thêm một dòng chỉ dẫn gắn cờ theo từng yêu cầu đưa micro-F1 lên 0,73. Bộ đánh giá định lượng là thứ làm lộ ra lỗi này; đọc mắt kết quả JSON thì khó thấy.

3.4 Test Design Agent và Boundary Value Analysis

Agent chỉ nhận yêu cầu đã duyệt. LLM sinh test case các loại positive, negative, boundary, error_guessing và alternative_flow, mỗi test case mang ID REQ-xxx-TC-yy. Prompt cấm tạo expected result vượt quá yêu cầu đã duyệt.

LLM hay quên ca biên, nên hệ thống bổ sung module boundary_analysis tắt định. Module nhận ràng buộc số tường minh (khoảng, tối thiểu, tối đa) viết bằng tiếng Việt, tiếng Anh hoặc ký hiệu. Từ mỗi ràng buộc, nó sinh ca biên hợp lệ và ca ngay ngoài biên, kèm trích dẫn ràng buộc nguồn. Ràng buộc không rõ thì không sinh (fail-closed). Với "Mật khẩu 8-20 ký tự" và "tối thiểu 18", module sinh 6 ca biên và âm bản đúng nhãn.

3.5 Playwright Generation Agent

3.5.1 DSL action rút gọn

LLM không viết mã Python trực tiếp. Nó trả về một PlaywrightPlan gồm danh sách action trong một DSL nhỏ (Bảng 3.4); mã được render từ plan qua template Jinja2. Cách này giới hạn những gì LLM có thể làm, cho phép chính sách repair so sánh hai plan theo từng action, và loại bỏ lỗi cú pháp.

Bảng 3.4 DSL action của PlaywrightPlan

Action	Trường dùng	Mã Playwright sinh ra
goto	arg = URL	page.goto(url)
fill	strategy, value, arg	<locator>.fill(arg)
click	strategy, value, role_name	<locator>.click()
expect_url	arg	expect(page).to_have_url(arg)
expect_visible	strategy, value, role_name	expect(<locator>).to_be_visible()
expect_text	strategy, value, arg	expect(<locator>).to_contain_text(arg)

Strategy là một trong role, label, placeholder, text, test_id, css. Với role, value là vai trò ARIA còn role_name là tên hiển thị. Prompt nhấn mạnh điểm này vì lỗi phổ biến nhất của LLM là đặt tên nút vào value.

3.5.2 Chính sách locator và grounding

Khi có nhiều locator ứng viên cho cùng phần tử, locator_policy chọn locator khớp đúng một phần tử, ưu tiên theo thứ tự role, label, placeholder, text, test_id, css như khuyến nghị của Playwright. Grounding đếm số phần tử khớp với mỗi locator trên trang thật và lưu thành GroundingRecord (ok khi khớp đúng 1).

Grounding tính chụp DOM một lần nên bỏ sót phần tử chỉ có sau điều hướng, ví dụ danh sách sản phẩm chỉ hiện sau khi đăng nhập. Hàm ground_flow đi theo luồng: ở mỗi bước, nó chờ locator gắn vào DOM, đếm số phần tử khớp, rồi mới thực hiện goto, fill hoặc click để đẩy trang sang trạng thái kế. Assertion không làm đổi trạng thái. Test case đăng nhập SauceDemo đạt grounding 4/4, kể cả .inventory_list chỉ xuất hiện sau đăng nhập.

3.5.3 Ngăn cảnh accessibility tree (nhánh DOM-aware)

Để trả lời RQ2, hàm plan nhận thêm tham số tùy chọn page_context. Khi có, prompt được nối thêm aria snapshot của trang đích lúc vừa mở, cắt tối đa 6.000 ký tự, kèm chỉ dẫn: với phần tử có trong snapshot thì dùng đúng role và tên như snapshot; phần tử xuất hiện sau điều hướng thì suy luận như thường. Khi không truyền page_context, prompt giữ nguyên như baseline. Unit test kiểm tra cả hai trường hợp.

3.6 Execution Agent

Execution Agent ghi script ra tệp, chạy trong tiến trình con với timeout 120 giây và thu stdout, stderr, ảnh chụp. Trạng thái được phân loại: passed khi tiến trình thoát mã 0 và in PASSED; failed khi stderr chứa AssertionError, TimeoutError hoặc expect(blocked khi quá timeout; các trường hợp còn lại là error. Agent không sửa test.

Template script cấu hình hai tham số chạy, áp dụng như nhau cho mọi script. Timeout của expect là 10 giây vì một số trang demo cần khoảng 5 giây mới hiện kết quả, đúng bằng mặc định 5 giây của Playwright. Chromium chạy với cờ --disable-http2. Lý do của cờ này trình bày ở mục 4.3.4.

3.7 Repair Agent và chính sách sửa có ràng buộc

Repair Agent nhận test lỗi kèm bằng chứng và tạo đề xuất sửa dạng plan mới. Mức rủi ro do repair_policy quyết định bằng cách so sánh plan cũ và plan mới theo từng action, không để LLM tự khai (Bảng 3.5). Mọi đề xuất đều có requires_approval = true; agent

không tự áp dụng. Ngân sách sửa mặc định là 2 lần cho một chuỗi thực thi; vượt ngân sách thì chuyển `BLOCKED_FOR_REVIEW`.

Bảng 3.5 Phân mức rủi ro của đề xuất sửa

Mức	Thay đổi phát hiện được	Xử lý
Low	Chỉ đổi locator	Đề xuất, người duyệt đơn giản
Medium	Đổi dữ liệu nhập, URL điều hướng, thêm bước	Bắt buộc xem bằng chứng và duyệt
High	Đổi assertion, đổi loại action, bỏ bước	Bắt buộc duyệt, gắn cờ thay đổi ngữ nghĩa
Prohibited	Sửa mã nguồn website, xóa bằng chứng, lặp vô hạn	Từ chối

Self-healing locator là đường sửa không cần LLM. Khi locator gãy, hệ thống quét DOM sống lấy các ứng viên, so khớp mờ với ý định của locator cũ, đếm số phần tử khớp, rồi chọn ứng viên duy nhất theo `locator_policy`. Không có ứng viên duy nhất thì trả về rỗng. Trong demo, locator cố ý sai placeholder: `Username` được chữa thành `role:textbox[Username]` với rủi ro Low, và vẫn chờ người duyệt.

3.8 Truy vết, cổng duyệt và giao diện

Traceability service dựng chuỗi Requirement → TestCase → Script → Execution → Repair từ các ID, tính độ phủ yêu cầu (tỉ lệ yêu cầu có ít nhất một test case), độ phủ thực thi, pass rate và liệt kê artifact mờ coi. Bảng 3.6 liệt kê các cổng duyệt.

Bảng 3.6 Các cổng phê duyệt

Cổng	Khi nào	Trạng thái hiện thực
AG-01	Yêu cầu bị gắn cờ mơ hồ	Có trên giao diện Streamlit
AG-02	Trước khi sinh script từ test case	Có trên giao diện Streamlit
AG-03	Đề xuất sửa đổi assertion	Có ở tầng policy, chưa có trên giao diện
AG-04	Đề xuất đổi expected result	Có ở tầng policy, chưa có trên giao diện
AG-05	Đề xuất bỏ hoặc đổi bước kiểm thử	Có ở tầng policy, chưa có trên giao diện

Giao diện Streamlit cho chọn backend (mock, local, cloud) và nguồn tài liệu (mẫu, dán, tải lên), chạy Requirement Agent, duyệt hoặc từ chối từng yêu cầu, sinh test case cho các yêu cầu đã duyệt, duyệt test case và xem bảng truy vết. Logic duyệt nằm trong `review.py`, tách khỏi mã Streamlit để kiểm thử offline.

3.9 Lớp trừu tượng LLM

Mọi agent gọi LLM qua giao diện LLMClient với một phương thức `structured_completion(system, user, schema)`. Có hai hiện thực: `OpenAILLMClient` gọi mọi endpoint tương thích OpenAI (OpenAI, DeepSeek, LM Studio, Ollama) bằng `base_url`, và `MockLLMClient` trả đối tượng dựng sẵn cho test. Tham số `--profile cloud` đọc `.env.cloud` thay cho `.env`, nên đổi giữa Gemma chạy local và DeepSeek chỉ cần một cờ dòng lệnh. LM Studio từ chối `response_format` kiểu `json_object`, nên hệ thống dùng `json_schema`, kiểu được cả ba backend chấp nhận.

3.10 Kiểm thử phần mềm

Bộ unit test gồm 83 test chạy offline bằng `MockLLMClient` và hàm đếm giả, không cần mạng hay trình duyệt. Các phần tất định được test kỹ: `repair policy`, `traceability`, `self-healing`, `boundary analysis`, `locator policy`, bộ đánh giá, `render script` và logic duyệt của giao diện. Phần cần trình duyệt thật (`grounding` đa bước, `chạy script`) được kiểm chứng bằng chạy thật trên `SauceDemo` và bằng oracle RQ2 ở Chương 4.

Chương 4: MÔI TRƯỜNG THỰC NGHIỆM VÀ PHƯƠNG PHÁP ĐÁNH GIÁ

4.1 Môi trường thực nghiệm

Thực nghiệm chạy trên một máy Windows 11, Python 3.14, Playwright 1.62 với Chromium headless. Backend cloud là DeepSeek (model deepseek-chat) qua API tương thích OpenAI. Backend local là google/gemma-4-12b chạy trên LM Studio với GPU 8 GB. Các website đích đều là site demo công khai dựng cho mục đích luyện kiểm thử tự động.

4.2 Đánh giá RQ1: phát hiện mơ hồ

Ground truth được tạo bằng cách chèn khuyết tật có chủ đích vào 5 tài liệu (auth, booking, cart, profile, search) với tổng 15 yêu cầu. Mỗi yêu cầu có nhãn loại mơ hồ biết trước. Một số yêu cầu được giữ sạch để đo over-flag. Đơn vị so khớp là cặp (yêu cầu, loại mơ hồ); yêu cầu dự đoán được căn với yêu cầu gốc theo thứ tự use case; mâu thuẫn được tính như một loại ở mức toàn cục. Chạy: python evaluate_agent.py (thêm -profile cloud cho DeepSeek).

4.3 Đánh giá RQ2: ngữ cảnh DOM và first-run pass rate

4.3.1 Catalog target

Catalog datasets/reference/rq2_targets.json gồm 50 target trên 6 website (Bảng 4.1), 37 positive và 13 negative. Mỗi target là một test case viết bằng tiếng Việt (các bước và expected_result) trên một URL thật. expected_result mô tả kết quả thật của trang để LLM có căn cứ sinh assertion; RQ2 đo khả năng chuyển test case đã duyệt thành mã, không đo khả năng đoán oracle. Danh sách đầy đủ ở Phụ lục A.

Bảng 4.1 Các website trong catalog RQ2

Website	Số target	Loại kịch bản
The Internet (Heroku)	16	Đăng nhập, JS alert/confirm, nội dung động, redirect, status code
SauceDemo	14	Đăng nhập, giỏ hàng, checkout, chi tiết sản phẩm, đăng xuất
UI Testing Playground	8	Form đăng nhập, đổi tên nút, đếm click, điều hướng
Practice Test Automation	5	Đăng nhập, đăng xuất, thêm dòng động
DemoQA	4	Form text box, radio button, nút bấm
OrangeHRM Demo	3	Đăng nhập và validation

4.3.2 Oracle viết tay: kiểm chứng target khả thi

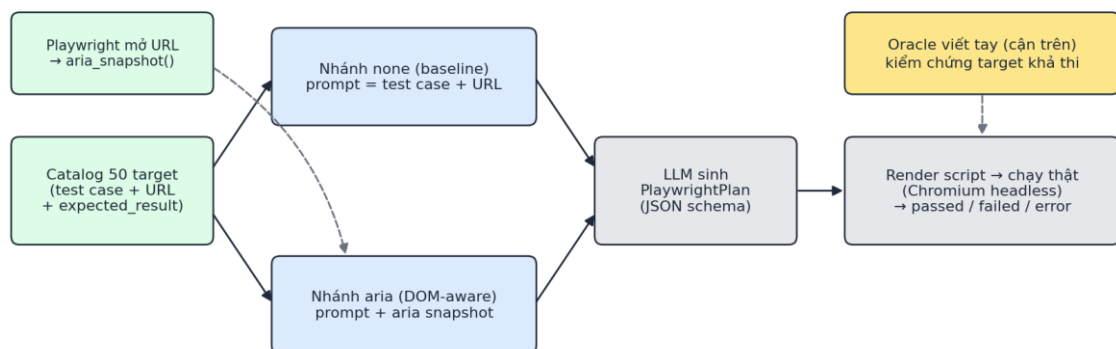
Một target có thể thất bại vì lý do không liên quan tới LLM: expected_result viết sai, site thay đổi, hoặc kịch bản cần thao tác ngoài DSL (chọn dropdown, hover, nhấn phím). Để loại các nguồn nhiễu này, mỗi target có một trường oracle là plan viết tay bằng đúng DSL mà LLM được dùng. Lệnh `python run_rq2.py --context oracle` chạy các plan này qua cùng template và Execution Agent.

Trong lúc xây catalog, oracle đã loại nhiều ứng viên. TodoMVC bị loại vì thêm việc cần nhấn Enter. Các trang AJAX và client delay của UI Testing Playground bị loại vì kết quả hiện sau khoảng 15 giây. Dropdown, hover, key press và double click bị loại vì DSL không có thao tác tương ứng. Oracle cũng bắt được lỗi trong catalog cũ: OrangeHRM hiển thị chữ "Required" dưới cả hai ô khi để trống cả hai, nên assertion text khớp hai phần tử và vi phạm strict mode. Target này được sửa thành bỏ trống riêng ô mật khẩu.

4.3.3 Hai nhánh so sánh

Cả hai nhánh dùng cùng model, cùng system prompt, cùng 50 test case và cùng template thực thi. Chỉ khác ở ngữ cảnh đưa thêm vào prompt (Hình 4.1):

- none (baseline): prompt gồm URL đích, tiêu đề, loại, các bước và expected_result của test case.
- aria (DOM-aware): prompt như baseline, thêm aria snapshot của trang đích chụp ngay trước khi gọi LLM.



Hình 4.1 Quy trình đo RQ2 trên hai nhánh

Script sinh ra được chạy đúng một lần, không qua Repair Agent hay sửa tay. Plan không hợp lệ (LLM trả JSON hỏng) được tính là error. Mỗi kết quả lưu kèm dòng lỗi cuối của stderr để phân loại nguyên nhân. Lệnh chạy: `python run_rq2.py --profile cloud --context both --out rq2_results_both.json`.

4.3.4 Kiểm soát nhiễu hạ tầng

Khi chạy oracle lần đầu, mọi target của The Internet đều fail ở `page.goto` do timeout 30 giây, trong khi curl tải đúng các tài nguyên đó dưới 1 giây. Theo dõi request cho thấy Chromium kẹt khi tải tệp js và png tĩnh qua HTTP/2 từ mạng của nhóm. Thêm cờ `--disable-http2` thì cả ba lần thử đều tải xong trong 3,3 giây. Lỗi này cũng có thể đã làm giảm kết quả của lần đo thử ngày 15/09 (8/19 target đạt), trong đó The Internet chỉ đạt 3/7.

Sau khi tắt HTTP/2, The Internet vẫn thỉnh thoảng treo khi tải trang. Đây là lỗi hạ tầng, không phải lỗi script, nên khi một script fail với thông điệp `Page.goto: Timeout`, hệ thống chạy lại đúng script đó một lần và đánh dấu `infra_retry` trong JSON. Script không đổi nên phép đo first-run vẫn giữ nguyên ý nghĩa: LLM chỉ có một cơ hội sinh mã.

4.4 Đánh giá RQ3 và RQ4

RQ3 hiện mới có kịch bản demo trên SauceDemo: một test case đúng và một test case cố ý sai locator, chạy qua toàn bộ pipeline bằng `run_full_pipeline.py`. RQ4 (thời gian và số thao tác thủ công tiết kiệm được) cần thí nghiệm có người tham gia và chưa được thực hiện trong chuyên đề này.

Chương 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 RQ1: phát hiện mơ hồ trong yêu cầu

Bảng 5.1 là kết quả trên 15 yêu cầu sau khi siết prompt. Dòng Gemma "chưa nắn prompt" là cấu hình trước khi thêm chỉ dẫn gắn cờ theo từng yêu cầu (mục 3.3).

Bảng 5.1 Kết quả phát hiện mơ hồ trên tập 15 yêu cầu

Model	Micro-P	Micro-R	Micro-F1	Macro-F1	Over-flag
deepseek-chat (cloud)	0,50	1,00	0,67	0,71	0,20
gemma-4-12b, chưa nắn prompt	0,06	0,08	0,07	0,17	0,00
gemma-4-12b (local)	0,71	0,83	0,77	0,69	0,00

Siết prompt (chỉ gắn cờ khi thiếu hoặc mơ hồ rõ ràng, ưu tiên không gắn khi phân vân ở precondition và outcome) giảm over-flag ở cả hai backend (Bảng 5.2). Đổi lại, recall của Gemma giảm từ 1,00 xuống 0,83, chủ yếu do bỏ sót missing_precondition; DeepSeek giữ recall 1,00.

Bảng 5.2 Ảnh hưởng của việc siết prompt (trước → sau)

Model	Micro-F1	Over-flag	Recall
gemma-4-12b (local)	0,73 → 0,77	0,40 → 0,00	1,00 → 0,83
deepseek-chat (cloud)	0,51 → 0,67	0,80 → 0,20	1,00 → 1,00

Bảng 5.3 F1 theo từng loại mơ hồ

Loại	deepseek-chat	gemma-4-12b
conflict	1,00	1,00
missing_actor	0,86	1,00
missing_expected_outcome	0,60	0,67
missing_precondition	0,50	0,00
underspecified_action	0,29	0,50
vague_quantifier	1,00	1,00

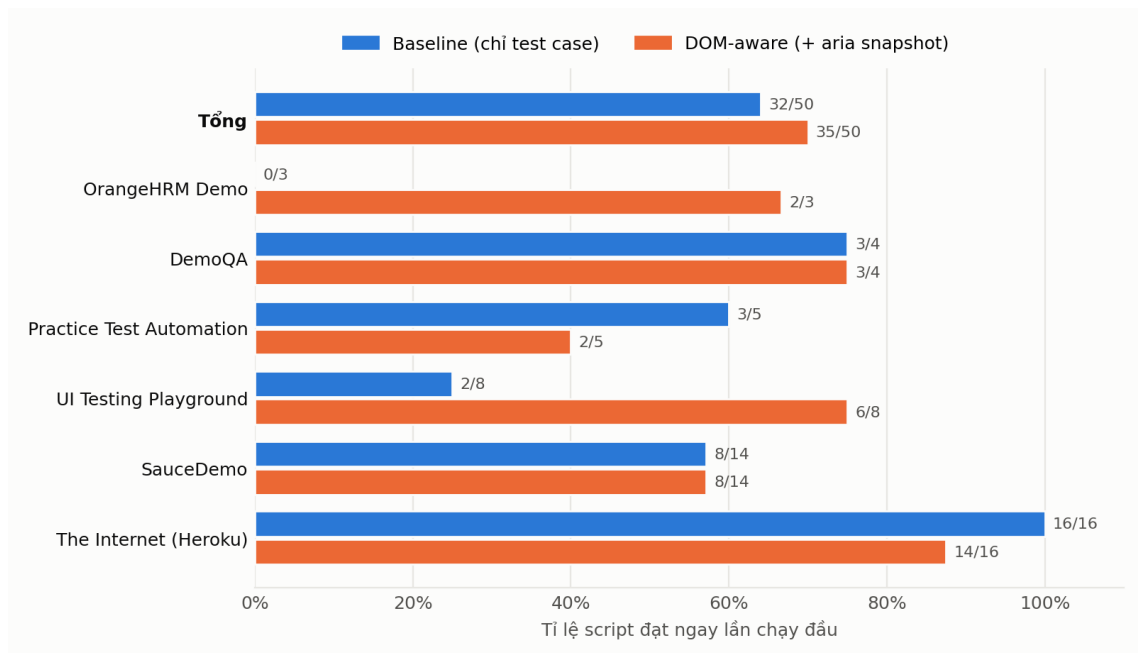
Dưới cùng một prompt, Gemma có precision và micro-F1 cao hơn, DeepSeek bắt đủ hơn (recall và macro-F1 cao hơn). missing_precondition là loại khó nhất với cả hai. Nhân gold của loại này cũng chủ quan nhất: mỗi yêu cầu chỉ có một khuyết tật được chèn, trong khi một người đọc khác có thể thấy thêm precondition còn thiếu. Với 15 yêu cầu, các con số này chỉ là sơ bộ; một yêu cầu đôi nhãn có thể làm F1 của một loại thay đổi vài chục điểm phần trăm.

5.2 RQ2: ngữ cảnh DOM và first-run pass rate

Oracle viết tay đạt 50/50 target, nên catalog khả thi với DSL hiện có và đóng vai càn trên của phép đo. Với model deepseek-chat, nhánh baseline đạt 32/50 (64%), nhánh có aria snapshot đạt 35/50 (70%).

Bảng 5.4 First-run pass rate theo website, hai nhánh

Website	Số target	Baseline (none)	DOM-aware (aria)
The Internet (Heroku)	16	16 (100%)	14 (88%)
SauceDemo	14	8 (57%)	8 (57%)
UI Testing Playground	8	2 (25%)	6 (75%)
Practice Test Automation	5	3 (60%)	2 (40%)
DemoQA	4	3 (75%)	3 (75%)
OrangeHRM Demo	3	0 (0%)	2 (67%)
Tổng	50	32 (64%)	35 (70%)



Hình 5.1 Tỷ lệ đạt lần đầu theo website, baseline và DOM-aware

Hai nhánh chạy trên cùng target nên có thể so sánh từng cặp (Bảng 5.5).

Bảng 5.5 Bảng chéo kết quả theo cặp target

	aria đạt	aria trượt
none đạt	29	3
none trượt	6	12

Có 6 target chỉ nhánh aria đạt và 3 target chỉ baseline đạt. Kiểm định McNemar chính xác trên 9 cặp bất đồng cho $p = 0.508$, không nhỏ hơn 0,05, nên với cỡ mẫu 50 chưa đủ bằng chứng thống kê để khẳng định khác biệt.

Bảng 5.6 Nguyên nhân thất bại theo nhánh

Nguyên nhân (từ dòng lỗi cuối của stderr)	Baseline	DOM-aware
Không tìm thấy phần tử để thao tác (timeout)	13	5
Locator khớp nhiều phần tử (strict mode)	5	9
Site không tải được (hạ tầng)	0	1
Tổng số target trượt	18	15

Trong 100 lần chạy có 1 lần cần chạy lại do Page.goto: Timeout (mục 4.3.4).

5.2.1 Lần đo thứ nhất: một lỗi đặc tả prompt bị che khuất

Lần đo đầu tiên cho baseline 21/50 (42%) và aria 38/50 (76%). Đọc từng script lỗi thì thấy 17 script baseline bắt đầu bằng `page.goto("")`, trong khi nhánh aria chỉ có 1. Gọi lại LLM trên ba target cho thấy DeepSeek đều đặt URL vào trường value thay vì arg. Nguyên nhân nằm ở system prompt: phần mô tả trường có nói fill dùng arg cho giá trị nhập và `expect_url` dùng arg cho URL, nhưng bỏ sót goto. Khi prompt dài thêm vì có aria snapshot, model lại điền đúng.

Nếu dừng ở lần đo này, khoảng cách giữa hai nhánh sẽ bị quy hết cho ngữ cảnh DOM, trong khi phần lớn nó đến từ một lỗi đặc tả không liên quan tới locator. Nhóm thêm một dòng `"goto: arg = URL"` vào prompt (áp dụng cho cả hai nhánh) rồi đo lại toàn bộ. Các kết quả ở mục 5.2 là của lần đo thứ hai. Bảng 5.7 đặt hai lần đo cạnh nhau.

Bảng 5.7 Hai lần đo RQ2 trước và sau khi sửa đặc tả goto

Lần đo	Baseline đạt	Aria đạt	goto rỗng (baseline / aria)
1 (prompt thiếu đặc tả goto)	21/50	38/50	17 / 1
2 (đã sửa prompt)	32/50	35/50	0 / 0

5.2.2 Thảo luận

Sau khi sửa đặc tả goto, nhánh aria đạt 35/50 so với 32/50 của baseline, hơn 3 target. Có 6 target chỉ aria đạt và 3 target chỉ baseline đạt, McNemar cho $p \approx 0,51$. Với 50 target và một lần chạy, nhóm chưa có bằng chứng thống kê rằng aria snapshot của trang đầu cải thiện first-run pass rate tổng thể.

Phân tích theo nguyên nhân cho thấy rõ hơn ngữ cảnh DOM đã làm gì. Nhóm lỗi "không tìm thấy phần tử để thao tác" giảm từ 13 xuống 5. Ví dụ điển hình là UI Testing Playground (2/8 lên 6/8) và OrangeHRM (0/3 lên 2/3). Không có snapshot, LLM đoán `get_by_label("Password")`, nhưng ô mật khẩu của Sample App không có

nhân, chỉ có placeholder "*****", còn nhân của OrangeHRM không gắn với ô nhập. Snapshot cho LLM thấy tên truy cập thật (textbox "*****", textbox "Username") nên nó viết `get_by_role` với đúng tên đó. Đây chính là loại lỗi mà grounding nhắm tới: locator trỏ tới một phần tử không tồn tại.

Nhóm lỗi "locator khớp nhiều phần tử" lại tăng từ 5 lên 9. Khi có snapshot, LLM dùng đúng tên nó thấy, ví dụ button "Add to cart", trong khi trang sản phẩm của SauceDemo có sáu nút cùng tên. Practice Test Automation in nguyên văn thông báo lỗi trong phần hướng dẫn trên trang, nên `get_by_text("Your username is invalid!")` khớp hai phần tử. Snapshot cho biết phần tử có tồn tại nhưng không cho biết locator có duy nhất hay không. DSL hiện tại cũng chưa có cách thu hẹp phạm vi (lọc theo khung sản phẩm, chọn phần tử thứ n). Ngữ cảnh DOM vì thế đổi loại lỗi từ "không tồn tại" sang "không duy nhất". Đây đúng là chỗ phân biệt locator validity với tính duy nhất trong GAP-03. Bước grounding đếm số phần tử khớp (mục 3.5.2) phát hiện được loại lỗi này trước khi chạy, nên hướng hợp lý là ghép hai cơ chế: snapshot cho LLM, grounding kiểm tính duy nhất, rồi trả kết quả đếm về cho LLM sinh lại.

Snapshot chỉ chụp trang đầu nên không giúp với phần tử xuất hiện sau điều hướng. Ở TI-15 và PT-04, nút đăng xuất chỉ có sau khi đăng nhập. Baseline đoán role link và đặt; nhánh aria đoán role button và trượt. Ba target chỉ baseline đạt đều rơi vào trường hợp này hoặc lỗi hạ tầng: TI-08 của nhánh aria trượt vì The Internet không tải được cả sau lần chạy lại. Snapshot theo từng bước, dựa trên `ground_flow`, là cải tiến trực tiếp cho điểm yếu này.

Bài học phương pháp từ lần đo thứ nhất đáng ghi lại. Lần đó cho 42% so với 76%, với 19 target chỉ aria đạt và 2 target chỉ baseline đạt ($p < 0,001$). Nhìn con số tổng, kết luận "ngữ cảnh DOM gần như gấp đôi tỉ lệ đạt" rất hấp dẫn, nhưng nó sai: phần lớn khoảng cách đến từ một dòng thiếu trong prompt. Hai việc giúp bắt được lỗi này là đọc từng script trượt thay vì chỉ đọc tỉ lệ, và có oracle chứng minh mọi target đều làm được, nên lỗi phải nằm ở phía sinh mã.

5.3 RQ3: sửa lỗi có ràng buộc (demo)

Chạy `run_full_pipeline.py` trên SauceDemo cho hai test case. TC-01 (đăng nhập hợp lệ) đạt grounding 3/3 locator khớp đúng một phần tử và chạy PASSED. TC-02 cố ý dùng locator sai placeholder:Username: grounding 2/3, chạy FAILED. Repair Agent

quét DOM sống và đề xuất `role:textbox[Username]`. Policy xếp đề xuất này mức Low vì chỉ đổi locator, đặt `requires_approval = true`, và agent không tự áp dụng. Báo cáo truy vết sinh tự động ghi độ phủ yêu cầu 100%, độ phủ thực thi 100%, pass rate 50% (một đạt, một lỗi có chủ đích).

Kịch bản này chứng minh cơ chế hoạt động đúng như thiết kế. Nó chưa đủ để trả lời RQ3: cần một tập lỗi lớn hơn gồm cả lỗi locator lẫn lỗi assertion, để đo tỉ lệ sửa thành công và tỉ lệ đề xuất làm đổi ngữ nghĩa bị chặn.

Chương 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được

- Xây dựng được pipeline năm agent chạy từ tài liệu yêu cầu tới script Playwright Python trên website thật, với các cổng duyệt AG-01, AG-02 trên giao diện Streamlit, truy vết từ yêu cầu tới đề xuất sửa, và 83 unit test chạy offline.
- RQ1 (sơ bộ, 15 yêu cầu): sau khi siết prompt, Gemma 4 12B chạy local đạt micro-F1 0,77 và không gán cò nhầm yêu cầu sạch nào; DeepSeek đạt micro-F1 0,67 với recall 1,00. Loại missing_precondition vẫn khó nhất.
- RQ2: catalog 50 target trên 6 website, oracle viết tay đạt 50/50. Với deepseek-chat, aria snapshot trang đầu cho 35/50 so với 32/50 của baseline, khác biệt chưa có ý nghĩa thống kê ($p \approx 0,51$). Ngữ cảnh DOM giảm lỗi phần tử không tồn tại (13 xuống 5) nhưng làm tăng lỗi locator không duy nhất (5 lên 9).
- Phát hiện và xử lý hai nguồn sai lệch trong phép đo: Chromium treo khi tải The Internet qua HTTP/2, và prompt thiếu đặc tả trường URL của goto. Lỗi thứ hai từng làm lần đo đầu thổi phồng hiệu ứng của ngữ cảnh DOM (42% so với 76%).
- Cơ chế sửa có ràng buộc hoạt động đúng thiết kế trên kịch bản demo: locator sai được chữa từ DOM sống, policy xếp mức rủi ro Low, đề xuất chờ người duyệt và không được tự áp dụng.

6.2 Hạn chế

- Tập RQ1 nhỏ (5 tài liệu, 15 yêu cầu), nhận gold do nhóm tự gán với một khuyết tật mỗi yêu cầu; kết quả chỉ là sơ bộ.
- RQ2 dùng 50 target trên site demo, phần lớn là luồng ngắn (1–5 bước). Ứng dụng nghiệp vụ thật có luồng dài và DOM lớn hơn nhiều, nên snapshot 6.000 ký tự có thể không đủ.
- Nhánh aria chỉ chụp trang đầu. Phần tử xuất hiện sau điều hướng (giỏ hàng, trang checkout) LLM vẫn phải đoán.
- Mỗi nhánh chạy một lần với một model; chưa đo độ dao động giữa các lần sinh và giữa các model.
- RQ3 mới có demo, RQ4 chưa đo. SQL Server và các cổng AG-03..05 trên giao diện chưa hoàn thành.

- DSL chưa có select, hover, nhấn phím, double click, nên các kịch bản cần những thao tác này nằm ngoài phạm vi đo.

6.3 Hướng phát triển

- Snapshot theo bước: kết hợp ground_flow để chụp aria snapshot sau mỗi lần điều hướng và đưa cho LLM sinh tiếp, thay vì chỉ trang đầu.
- Chạy RQ2 nhiều lần và trên nhiều model (Gemma local, DeepSeek, GPT) để đo độ dao động và kiểm định chặt hơn.
- Xây tập lỗi có chủ đích cho RQ3 (đổi locator, đổi nhãn, đổi luồng) và đo tỉ lệ sửa thành công cùng tỉ lệ thay đổi ngữ nghĩa bị chặn.
- Thí nghiệm có người tham gia cho RQ4: so thời gian và số lần sửa tay giữa quy trình thủ công và pipeline.
- Lưu trữ bên bằng SQL Server, đưa AG-03..05 lên giao diện, mở rộng DSL.
- Mở rộng và cho nhiều người gán nhãn độc lập tập RQ1 để đo độ đồng thuận.

TÀI LIỆU THAM KHẢO

- [1] Microsoft. Playwright Test Agents. <https://playwright.dev/docs/test-agents>, 2025.
- [2] Microsoft. Playwright: Codegen và Best Practices. <https://playwright.dev/docs/best-practices>, 2026.
- [3] Z. Yang, R. Huang, C. Cui, N. Niu, D. Towey. Requirements-Based Test Generation: A Comprehensive Survey. arXiv:2505.02015, 2025.
- [4] X. Teoh, Y. Lin, D.-M. Nguyen, R. Ren, W. Zhang, J. S. Dong. WebTestPilot. FSE 2026, arXiv:2602.11724, DOI 10.1145/3797115.
- [5] A. M. Sami, Z. Rasheed, M. Waseem, Z. Zhang, T. Herda, P. Abrahamsson. A Tool for Test Case Scenarios Generation Using Large Language Models. arXiv:2406.07021, 2024.
- [6] R. N. Joseph. Beyond LLM-Based Test Automation: Zero-Cost Self-Healing via DOM Accessibility Tree. arXiv:2603.20358, 2026.
- [7] S. Masuda, S. Kouzawa, K. Sezai, H. Suhara, Y. Hiruta, K. Kudou. Generating High-Level Test Cases from Requirements using LLM: An Industry Study. arXiv:2510.03641, 2025.
- [8] H. Lee. Practical Limits of Autonomous Test Repair: A Multi-Agent Case Study. arXiv:2605.01471, 2026.
- [9] T. Schlathölter. ReqToCode: Embedding Requirements Traceability as a Structural Property of the Codebase. arXiv:2603.13999, 2026.
- [10] B. Tang, K. G. Shin. Steward: Natural Language Web Automation. arXiv:2409.15441, 2024.
- [11] S. Bhatia, T. Gandhi, D. Kumar, P. Jalote. System Test Case Design from Requirements Specifications: Insights and Challenges of Using ChatGPT. arXiv:2412.03693, 2024.
- [12] A. Kluge, A. Stocco. Web Element Relocalization in Evolving Web Applications. arXiv:2505.16424, 2025.

PHỤ LỤC A: DANH SÁCH 50 TARGET RQ2

Bảng A.1 Catalog target RQ2 (datasets/reference/rq2_targets.json)

ID	Website	Loại	Các bước	Kết quả mong đợi
SD-01	SauceDemo	positive	Nhập username 'standard_user' và password 'secret_sauce'; Bấm nút Login	Đăng nhập thành công, URL là https://www.saucedemo.com/inventory.html
SD-02	SauceDemo	negative	Nhập username 'standard_user' và password 'wrong_password'; Bấm Login	Hiện thông báo lỗi chứa 'Username and password do not match'
SD-03	SauceDemo	negative	Nhập username 'locked_out_user' và password 'secret_sauce'; Bấm Login	Hiện thông báo chứa 'Sorry, this user has been locked out'
SD-04	SauceDemo	negative	Không nhập gì; Bấm nút Login	Hiện thông báo chứa 'Username is required'
SD-05	SauceDemo	negative	Nhập username 'standard_user', bỏ trống password; Bấm Login	Hiện thông báo chứa 'Password is required'
SD-06	SauceDemo	positive	Nhập username 'problem_user' và password 'secret_sauce'; Bấm Login	Đăng nhập thành công, URL là https://www.saucedemo.com/inventory.html
SD-07	SauceDemo	positive	Nhập username 'performance_glitch_user' và password 'secret_sauce'; Bấm Login	Đăng nhập thành công, URL là https://www.saucedemo.com/inventory.html
TI-01	The Internet (Heroku)	positive	Nhập username 'tomsmith' và password 'SuperSecretPassword!'; Bấm nút Login	Hiện thông báo chứa 'You logged into a secure area!'
TI-02	The Internet (Heroku)	negative	Nhập username 'baduser' và password 'badpass'; Bấm Login	Hiện thông báo chứa 'Your username is invalid!'
TI-03	The Internet (Heroku)	positive	Bấm nút 'Add Element'	Xuất hiện nút 'Delete' trên trang
TI-04	The Internet (Heroku)	positive	Bấm nút 'Start'	Sau khi tải xong hiện văn bản chứa 'Hello World!'
TI-05	The Internet (Heroku)	positive	Bấm nút 'Start'	Sau khi tải xong hiện văn bản chứa 'Hello World!'
TI-06	The Internet (Heroku)	positive	Mở trang	Trang hiện văn bản chứa 'This page returned a 200 status code'
TI-07	The Internet	positive	Mở trang	Trang hiện văn bản chứa

ID	Website	Loại	Các bước	Kết quả mong đợi
	(Heroku)			'This page returned a 404 status code'
PT-01	Practice Test Automation	positive	Nhập username 'student' và password 'Password123'; Bấm nút Submit	Hiện thông báo chứa 'Logged In Successfully'
PT-02	Practice Test Automation	negative	Nhập username 'incorrectUser' và password 'Password123'; Bấm Submit	Hiện thông báo chứa 'Your username is invalid!'
PT-03	Practice Test Automation	negative	Nhập username 'student' và password 'wrongPassword'; Bấm Submit	Hiện thông báo chứa 'Your password is invalid!'
OH-01	OrangeHRM Demo	positive	Nhập username 'Admin' và password 'admin123'; Bấm nút Login	Đăng nhập thành công, hiện tiêu đề 'Dashboard'
OH-02	OrangeHRM Demo	negative	Nhập username 'Admin' và password 'wrongpass'; Bấm Login	Hiện thông báo chứa 'Invalid credentials'
TI-08	The Internet (Heroku)	positive	Bấm nút 'Click for JS Alert' (hộp thoại alert được đóng)	Mục Result hiện văn bản chứa 'You successfully clicked an alert'
TI-09	The Internet (Heroku)	positive	Bấm nút 'Click for JS Confirm'; Hộp thoại xác nhận bị đóng (tương đương Cancel)	Mục Result hiện văn bản chứa 'You clicked: Cancel'
TI-10	The Internet (Heroku)	positive	Bấm nút 'Remove' ở mục Remove/add	Sau khi xử lý xong hiện thông báo chứa "It's gone!"
TI-11	The Internet (Heroku)	positive	Bấm nút 'Enable' ở mục Enable/disable	Sau khi xử lý xong hiện thông báo chứa "It's enabled!"
TI-12	The Internet (Heroku)	positive	Mở trang	Trang hiện văn bản chứa 'This page returned a 500 status code'
TI-13	The Internet (Heroku)	negative	Để trống username và password; Bấm nút Login	Hiện thông báo chứa 'Your username is invalid!'
TI-14	The Internet (Heroku)	negative	Nhập username 'tomsmith' và password 'wrongpass'; Bấm nút Login	Hiện thông báo chứa 'Your password is invalid!'
TI-15	The Internet (Heroku)	positive	Nhập username 'tomsmith' và password 'SuperSecretPassword!'; Bấm nút Login; Ở trang Secure Area bấm nút 'Logout'	Quay về trang login và hiện thông báo chứa 'You logged out of the secure area!'
TI-16	The Internet (Heroku)	positive	Bấm liên kết 'here'	Bị chuyển hướng, URL là https://the-internet.herokuapp.com/status/500

ID	Website	Loại	Các bước	Kết quả mong đợi
				tus_codes
SD-08	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Bấm 'Add to cart' ở sản phẩm 'Sauce Labs Backpack'	Biểu tượng giỏ hàng hiện số '1'
SD-09	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Bấm 'Add to cart' ở sản phẩm 'Sauce Labs Backpack'; Bấm 'Remove' ở cùng sản phẩm đó	Nút 'Add to cart' của 'Sauce Labs Backpack' hiện lại
SD-10	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Bấm vào tên sản phẩm 'Sauce Labs Backpack'	Mở trang chi tiết sản phẩm, mô tả chứa 'carry.allTheThings()'
SD-11	SauceDemo	negative	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Thêm 'Sauce Labs Backpack' vào giỏ; Mở giỏ hàng, bấm 'Checkout'; Bỏ trống First Name, nhập Last Name 'Nguyen' và Zip '70000'; Bấm 'Continue'	Hiện thông báo lỗi chứa 'Error: First Name is required'
SD-12	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Thêm 'Sauce Labs Backpack' vào giỏ; Mở giỏ hàng, bấm 'Checkout'; Nhập First Name 'An', Last Name 'Nguyen', Zip '70000', bấm 'Continue'; Bấm 'Finish'	Hiện tiêu đề 'Thank you for your order!'
SD-13	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Mở menu (nút 'Open Menu'); Bấm 'Logout'	Quay về trang đăng nhập, URL là https://www.saucedemo.com/
SD-14	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Bấm biểu tượng giỏ hàng	Mở trang giỏ hàng, tiêu đề chứa 'Your Cart'
UP-01	UI Testing Playground	positive	Nhập User Name 'tester' và password 'pwd'; Bấm nút 'Log In'	Hiện trạng thái chứa 'Welcome, tester!'
UP-02	UI Testing Playground	negative	Nhập User Name 'tester' và password 'wrong'; Bấm nút	Hiện trạng thái chứa 'Invalid'

ID	Website	Loại	Các bước	Kết quả mong đợi
			'Log In'	username/password'
UP-03	UI Testing Playground	positive	Nhập User Name 'tester' và password 'pwd'; Bấm nút 'Log In'; Bấm nút 'Log Out'	Hiện trạng thái chứa 'User logged out.'
UP-04	UI Testing Playground	positive	Nhập 'Hello' vào ô 'Set New Button Name'; Bấm nút bên dưới ô nhập	Nút đổi tên thành 'Hello'
UP-05	UI Testing Playground	positive	Mở trang	Ở mục Playground hiện văn bản chứa 'Welcome UserName!'
UP-06	UI Testing Playground	positive	Bấm liên kết 'Click me' một lần	Bộ đếm hiện 'The link above clicked 1 times.'
UP-07	UI Testing Playground	positive	Ở trang chủ bấm liên kết 'Sample App'	Chuyển tới trang Sample App, URL là http://uitestingplayground.com/sampleapp
UP-08	UI Testing Playground	positive	Ở trang chủ bấm liên kết 'Dynamic ID'	Chuyển tới trang có tiêu đề 'Dynamic ID'
DQ-01	DemoQA	positive	Nhập Full Name 'Nguyen Van A' và Email 'a@example.com'; Bấm nút 'Submit'	Khung kết quả hiện 'Name:Nguyen Van A'
DQ-02	DemoQA	positive	Chọn radio 'Yes'	Hiện văn bản 'You have selected Yes'
DQ-03	DemoQA	positive	Chọn radio 'Impressive'	Hiện văn bản 'You have selected Impressive'
DQ-04	DemoQA	positive	Bấm (click đơn) nút 'Click Me'	Hiện văn bản 'You have done a dynamic click'
PT-04	Practice Test Automation	positive	Nhập username 'student' và password 'Password123'; Bấm nút Submit; Bấm nút 'Log out'	Quay về trang login, URL là https://practicetestautomation.com/practice-test-login/
PT-05	Practice Test Automation	positive	Bấm nút 'Add'	Hiện thông báo xác nhận chứa 'Row 2 was added'
OH-03	OrangeHRM Demo	negative	Nhập username 'Admin', bỏ trống password; Bấm nút Login	Hiện thông báo kiểm tra hợp lệ 'Required' dưới ô Password